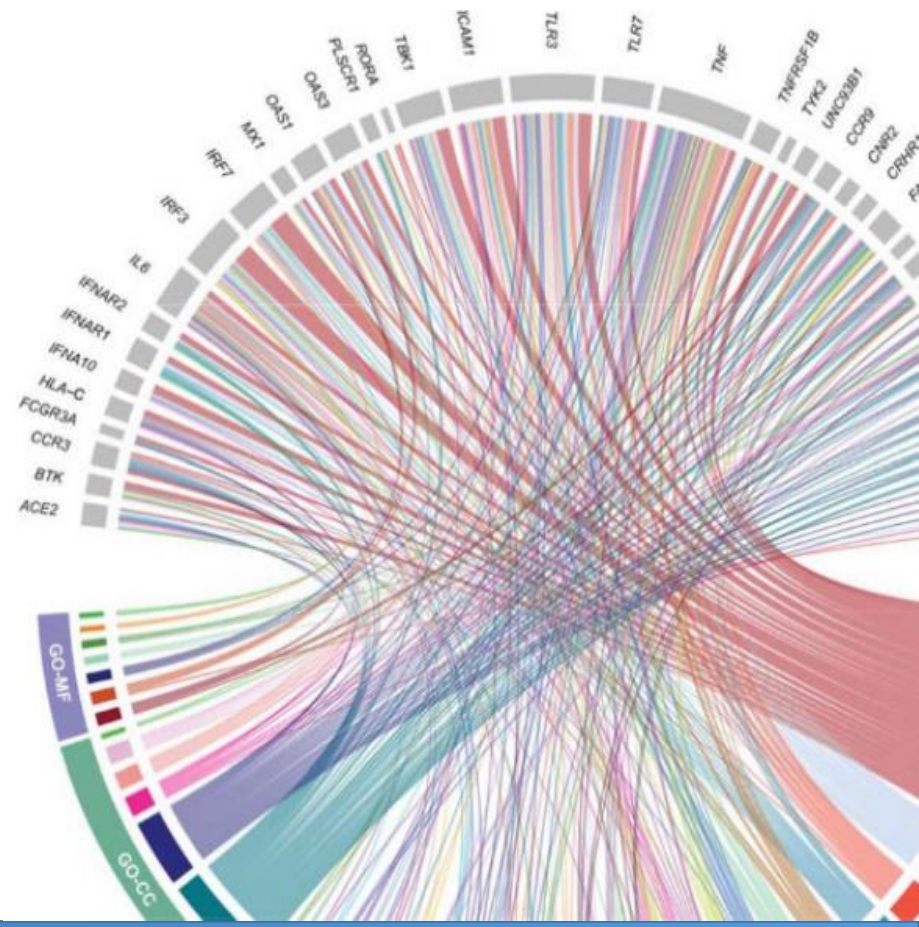
Tècniques i Eines Bioinformàtiques

Sequence Alignment

Santiago Marco-Sola (santiago.marco@upc.edu)

Màster en Enginyeria Informàtica, UPC
Departament of Computer Science
Facultat d'Informàtica de Barcelona (FIB), UPC



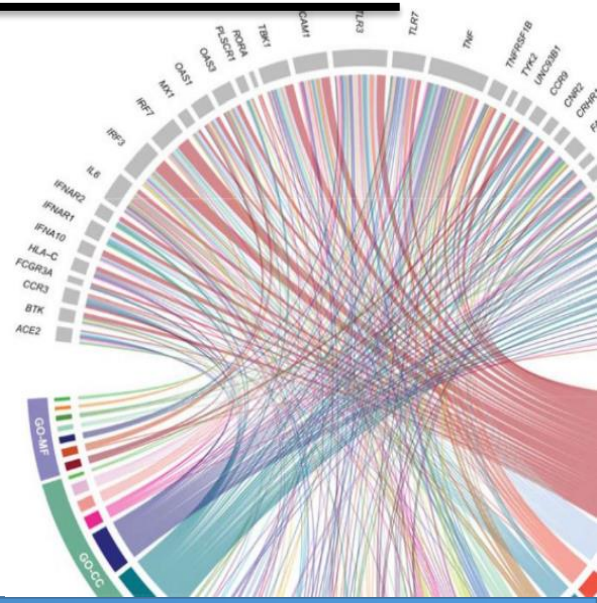
Acknowledgements

Many pictures and materials are taken from **Ben Langmead's course**.

Course heavily inspired in:

- **Genome-Scale Algorithm Design**. Veli Mäkinen, Djamal Belazzougui, Fabio Cunial, Alexandru I. Tomescu. Cambridge University Press.
- **Algorithms on Strings, Trees, and Sequences**. Dan Gusfield. Cambridge University Press.
- **An Introduction to Bioinformatics Algorithms.** Neil C. Jones, Pavel A. Pevzner. MIT Press.

1 Sequence Alignment



Approximate Matching for Biosequences

- Some widely used approximate matching algorithms for DNA (& other strings) came from the biological community, aiming to bring alignments into closer harmony with biological processes that mutate strings

```
Score = 248 bits (129), Expect = 1e-63
Identities = 213/263 (80%), Gaps = 34/263 (12%)
Strand = Plus / Plus

Query: 161 atatcaccacgtcaaaggtgactccaactcca---ccactccattttgttcagataatgc 217
          |||||||||||||||||||||||||||||  |     | |   || |||||||||||||
Sbjct: 481 atatcaccacgtcaaaggtgactccaact-tattgatagtgttttatgttcagataatgc 539

Query: 218 ccgatgatcatgtcatgcagctccaccgattgtgagaacgacagcgacttccgtcccagc 277
          |||||||   |||||||||||||||||||||| || |      |||||||||||
Sbjct: 540 ccgatgactttgtcatgcagctccaccgattttg-g------------ttccgtcccagc 586
Query: 278 c-gtgcc--aggtgctgcctcagattcaggttatgccgctcaattcgctgcgtatatcgc 334
          | || ||  |||||||||||||||||||||||||||||||||||||   |||||||||
Sbjct: 587 caatgacgta-gtgctgcctcagattcaggttatgccgctcaattcgctgggtatatcgc 645

Query: 335 ttgctgattacgtgcagctttcccttcaggcggga------------ccagccatccgtc 382
          |||||||||||||||||||||||||||||||||||      |||||||||||||
Sbjct: 646 ttgctgattacgtgcagctttcccttcaggcgggattcatacagcggccagccatccgtc 705
```

Pairwise Sequence Alignment

- In sequence comparison, pairwise alignment compares two sequences
  a) Determine its similarity (i.e. distance or **score**).
  b) Compute the **alignment** between both.
    - How to convert one sequence into the other applying a series of error events or alignment operations that minimize a given cost function or distance function
- Applications
  - **Computational biology** (e.g. genome biology, protein comparisons, evolutionary studies)
  - Information Retrieval (e.g. diff linux tool, wikipedia, git)
  - Signal processing and sound recognition
  - Others: Compression, pattern recognition, writing recognition, …

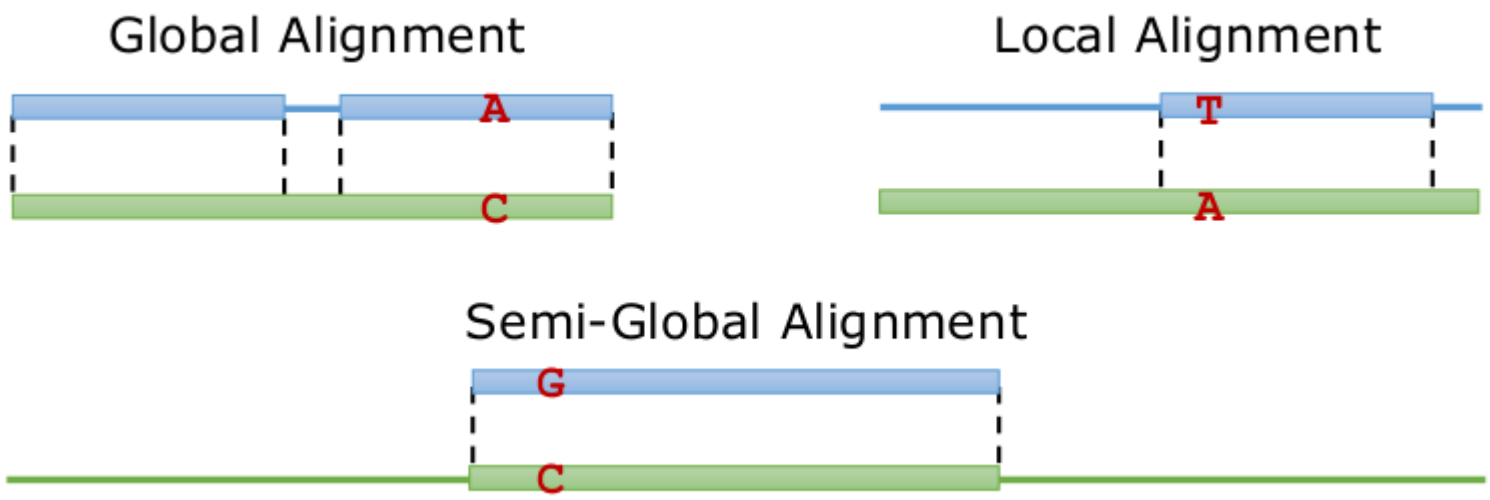
Distance Metrics

- Some **distance metrics** are best suited to model some application problems
  - Mismatch Distance (a.k.a. Hamming distance or Manhattan distance)
  - Edit Distance (a.k.a. Levenshtein distance)
  - Linear-Gap Distance (Needleman-Wunch and Smith-Waterman algorithms)
  - Affine-Gap Distance (Smith-Waterman-Gotoh algorithm)

- Different distance metrics involve different **alignment operations**.
  - Mismatch distance = {mismatches} = {M,X}
  - **Levenshtein distance** = {mismatches,insertions,deletions} = **{M,X,I,D}**
  - Episodic distance = {insertions} = {M,I}
  - Indel distance (LCS distance) = {insertions,deletions} = {M,I,D}
  - Damerau–Levenshtein distance = {mismatches,insertions,deletions,transpositions} = {M,X,I,D,T}

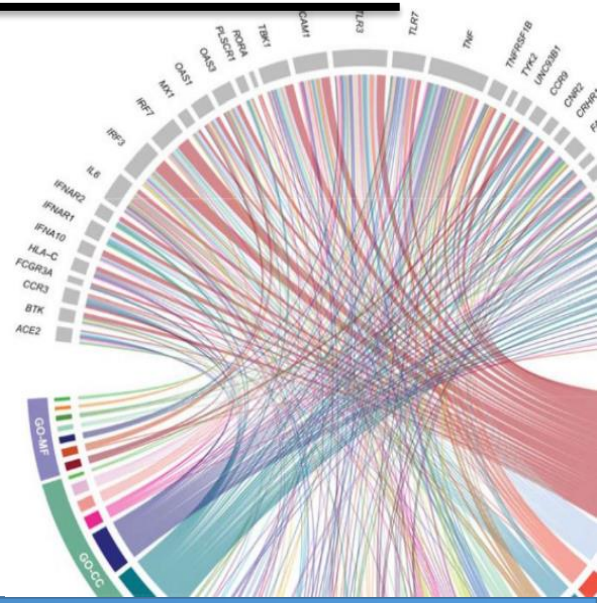
Alignment type

- Depending on the application, different types of alignment can be used.
  - **Global alignment**. Aligns both sequences end-to-end.
  - **Local alignment**. Locates a region that aligns between both sequences with highest similarity (i.e. highest score).
  - **Semi-global alignment**. Mostly applied to sequences very different in length (e.g. sequence against a genome).



2

Edit Distance/Alignment



Edit Distance

Imagine edits are introduced by an *optimal editor* working left-to-right:

```
x: G C G T A T G C G G C T A A C G C

y: G C T A T G C G G C T A T A C G C

x: G C G T A T G C G G C T A A C G C
   | |
y: G C - T A T G C G G C T A T A C G C

x: G C G T A T G C G G C T A - A C G C
   | |   | | | | | | | | | |
y: G C - T A T G C G G C T A T A C G C

x: G C G T A T G C G G C T A - A C G C
   | |   | | | | | | | | | |   | | | |
y: G C - T A T G C G G C T A T A C G C
```

Edit transcript summarizes how editor turns *x* into *y*

Operations:

M = match, **R** = replace,

I = insert into *x*, **D** = delete from *x*

MMD

Reminder: this is **D**, not **I**, because we have to delete a character from *x* to make it more like *y*

MMDMMMMMMMMMMI

MMDMMMMMMMMMMIMMMM

Edit Distance

Alignments:

Edit transcripts (w/r/t *x*):

```
x: GCGTATGCGGCTA-ACGC
   ||  |||||||||||  ||||
y: GC-TATGCGGCTATACGC
```

MMDMMMMMMMMMMIMMMM

Distance = 2

replacement (AKA substitution/mismatch)

```
x: GCGTATGAGGCTA-ACGC
   ||  ||||  ||||||  ||||
y: GC-TATGCGGCTATACGC
```

MMDMMMMXMMMMMIMMMM

Distance = 3

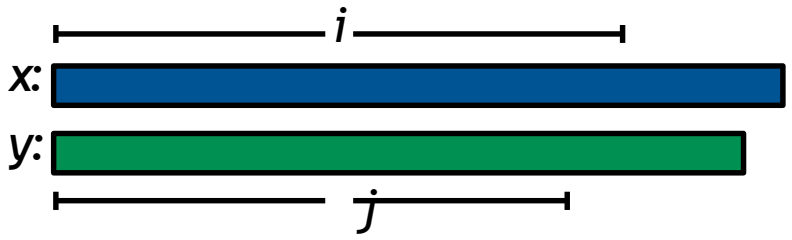
```
x: the  longest----
       |||||||
y: ----longest  day
```

DDDDMMMMMMMIIII

Distance = 8

Edit Distance

D[i, j]: edit distance between length-i prefix of x and length-j prefix of y



Optimal edit transcript for D[i, j] is built by extending a shorter one by 1 operation. 3 options:

Append **D** to transcript for D[i-1, j]Append **I** to transcript for D[i, j-1]

Append **M** or **X** to transcript for D[i-1, j-1]

D[i, j] and its transcript come from 1 of these 3 choices, whichever has fewest edits D[|x|, |y|] is the overall edit distance

Edit Distance

- Optimization problem:
  - Compute the **minimum** number of operations (match, substitution, insertion, and deletion) that transforms one sequence into another.
  - Each edit operation (substitution, single insertion, or single deletion) has unitary cost.

```
pattern ──→ GATTACA
            || |  |
text    ──→ GAAT  A
```

```python
def edit_distance(pattern,text):
    if len(pattern) == 0:
        # Insert all remaining chars
        return len(text)
    elif len(text) == 0:
        # Delete all remaining chars
        return len(pattern)
    else:
        # Find score for match/subtitution
        if pattern[0] == text[0]:
            m_cost = edit_distance(pattern[1:],text[1:])
        else:
            m_cost = edit_distance(pattern[1:],text[1:]) + 1
        # Find score for insertion
        i_cost = edit_distance(pattern[:],text[1:]) + 1
        # Find score for deletion
        d_cost = edit_distance(pattern[1:],text[:]) + 1
        # Find the minimum combination
        return min(m_cost,i_cost,d_cost)

distance = edit_distance("GATTACA","GAATA")
```

Edit Alignment

- Compute the **actual set of operations** (match, substitution, insertion, and deletion) that minimizes distance/cost function.
  - Multiple solutions possible
  - Algorithms finds the “first-best”

```
GATTACA
||  |   |
GAAT   A
```

MMXMDDM

CIGAR

2MXM2DM

```python
def edit_distance(pattern,text):
    if len(pattern) == 0:
        return (len(text),["I"] * len(text))
    elif len(text) == 0:
        return (len(pattern),["D"] * len(pattern))
    else:
        (m_cost,m_cigar) = edit_distance(pattern[1:],text[1:])
        (i_cost,i_cigar) = edit_distance(pattern[:],text[1:])
        (d_cost,d_cigar) = edit_distance(pattern[1:],text[:])
        minimum = min(m_cost,i_cost,d_cost)
        if minimum == m_cost:
            if (pattern[0] == text[0]):
                return (m_cost,["M"] + m_cigar)
            else:
                return (m_cost+1,["X"] + m_cigar)
        elif minimum == i_cost:
            return (i_cost+1,["I"] + i_cigar)
        else:
            return (d_cost+1,["D"] + d_cigar)

distance = edit_distance("GATTACA","GAATA")
```

Edit Distance. Repeated calls

- Previous algorithm was based on **exploring the whole space of solutions** (alignments), in order to find the one(s) with minimum cost ~ $O(3^n)$
  - There are NOT so many combinations
  - Recursive calls repeat combinations

```
edit_distance('CA', '')
edit_distance('ACA', '')
edit_distance('A', '')
edit_distance('CA', '')
edit_distance('', '')
edit_distance('A', '')
edit_distance('', 'A')
edit_distance('A', 'A')
edit_distance('CA', 'A')
edit_distance('ACA', 'A')
edit_distance('ACA', '')
[...]
```

Total Calls	Parameters
1970	('A', '')
1666	('', 'A')
1289	('A', 'A')
1289	('', '')
1002	('CA', '')
681	('CA', 'A')
462	('', 'TA')
450	('ACA', '')
377	('A', 'TA')
321	('ACA', 'A')
231	('CA', 'TA')
170	('TACA', '')
129	('TACA', 'A')
129	('ACA', 'TA')

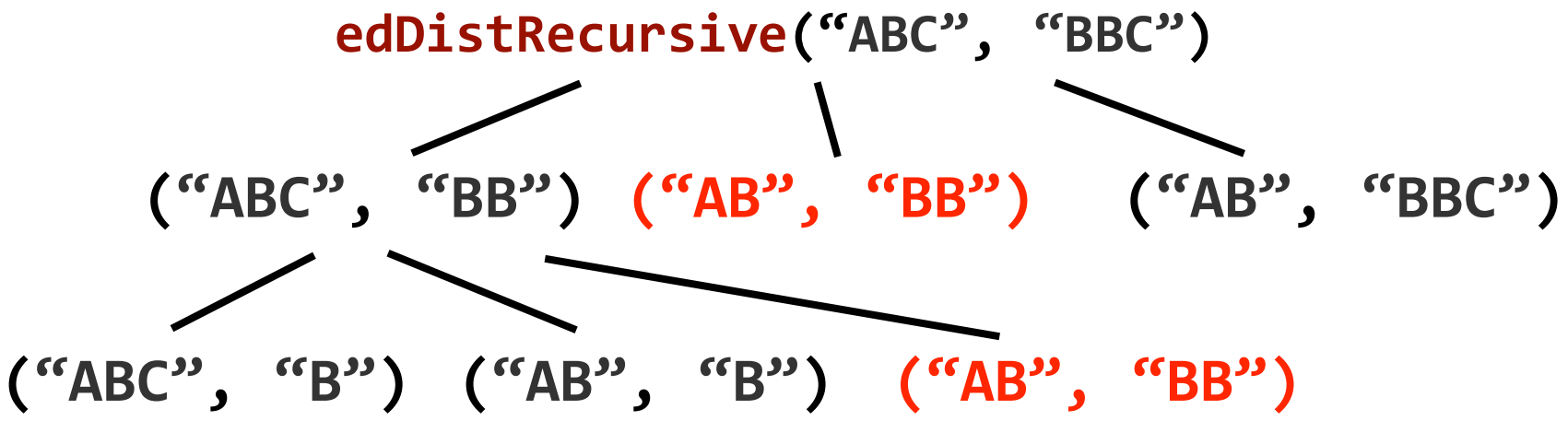
[...]

```
edDistRecursive("ABC", "BBC")
```

("ABC", "BB") ("AB", "BB") ("AB", "BBC")

("ABC", "B") ("AB", "B") ("AB", "BB")

(only part of recursion tree shown)



Some subtrees are identical!

(only part of recursion tree shown)

Memoization

- Use a memorization table (e.g. Hash Table) to store the **partial results** from computing the **edit distance of suffixes**.

1) Stored at the end

2) Checked at the beginning

```
def edit_distance(pattern,text,calls):
    # Lookup call
    key = pattern+":"+text
    if key in calls:
        return calls[key]
    # Regular algorithm
    if len(pattern) == 0:
        return len(text)
    elif len(text) == 0:
        return len(pattern)
    else:
        if pattern[0] == text[0]:
            m_cost = edit_distance(pattern[1:],text[1:])
        else:
            m_cost = edit_distance(pattern[1:],text[1:]) + 1
        i_cost = edit_distance(pattern[:],text[1:]) + 1
        d_cost = edit_distance(pattern[1:],text[:]) + 1
        minimum = min(m_cost,i_cost,d_cost)
        # Store call
        calls[pattern+":"+text] = minimum
        # Return
        return minimum
```

Dynamic Programming

- Observe that recursive calls are just computing the edit distance between suffixes of the pattern and suffixes of the text. Therefore we can define:

  $M_e(i,j)$ as the edit distance between the suffix pattern[i:] and text[j:]

  M_e(len(pattern),len(text)) as the **global alignment distance**

- There are only **n x m** combinations
  - Can be conveniently stored in a n x m matrix (i.e. **Dynamic Programming Table**)

$$\mathcal{M}_e(i,j) = \begin{cases} \max(i,j) & \text{if } i = 0 \vee j = 0, \\ \min \begin{cases} \mathcal{M}_e(i-1,j)+1 \\ \mathcal{M}_e(i,j-1)+1 \\ \mathcal{M}_e(i-1,j-1)+(P_i \neq T_j) \end{cases} & \text{otherwise} \end{cases}$$

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3

Dynamic Programming

- Each cell (i,j) encodes the edit distance between the suffix pattern[0,i-1] and text[0,j-1]
- First row and column contain the **initial conditions**
  - Row -> Cost of deleting leading text
  - Column-> Cost of deleting leading pattern
- Variations, e.g.
  - Semi-global alignment (ends-free alignment) sets the first row to zero to allow staring the alignment at any point in the text.

edit_distance('GATT', 'GAA') = $M_e(4,3)$

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3

edit_distance('GATTACA', 'GAATA') = $M_e(7,5)$

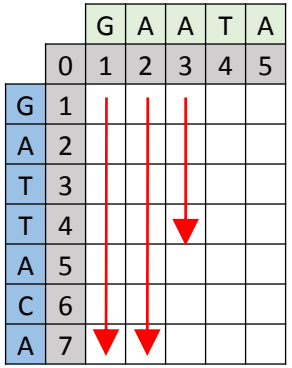
Dynamic Programming (Matrix Computation)

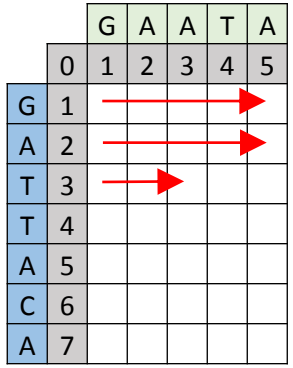
edit_distance('GAT', 'GAA') = $M_e(3,3)$
edit_distance('GAT', 'GA') = $M_e(3,2)$
edit_distance('GATT', 'GA') = $M_e(4,3)$

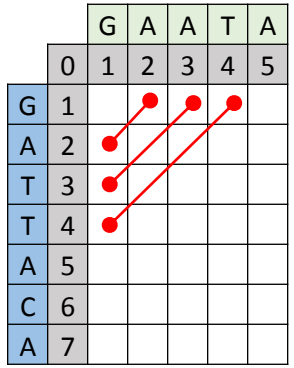
- Each DP-cell depends on other 3 cells. E.g.

$$M_e(4,3) \leftarrow \{M_e(4,3), M_e(3,3), M_e(3,2)\}$$

- The DP-table can be computed **column-wise, row-wise** or **antidiagonal-wise**. From the left top corner to the right bottom corner (i.e. solution).







		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3

edit_distance('GATT', 'GAA') = $M_e(4,3)$

Dynamic Programming (Backtrace Computation)

- "Backtrace" traces back the path with minimum cost that lead to the solution (i.e. minimum distance).
  - Linear cost, from the bottom right corner to the upper left corner.
  - Check which of the adjacent cells brought us to that cell
  - Quite a branchy code

```
GATTACA
||  |   |
GAAT   A
```

		G	A	A	T	A
	0	1	2	3	4	5
G	1	0	1	2	3	4
A	2	1	0	1	2	3
T	3	2	1	1	1	2
T	4	3	2	2	1	2
A	5	4	3	2	2	1
C	6	5	4	3	3	2
A	7	6	5	4	4	3

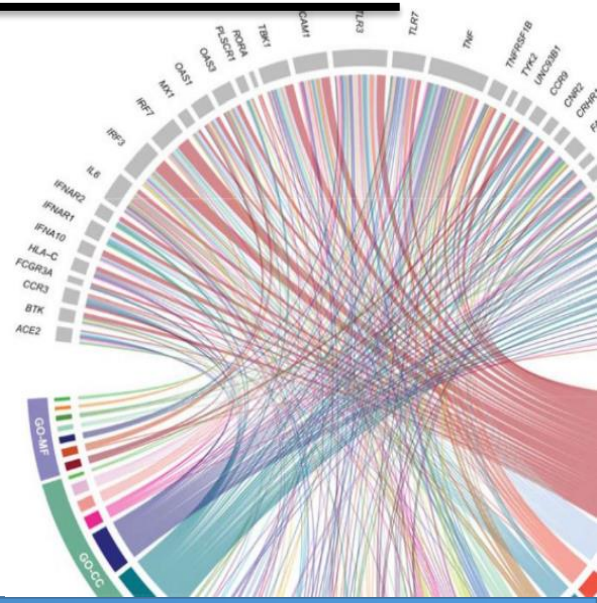
CIGAR MMXMDDM

Sequence Alignment Costs

Matrix-filling dynamic programming algorithm is $O(mn)$ time and space

- Filling matrix is $O(mn)$ space and time, yields global alignment value
- Traceback is $O(m + n)$ time, yields optimal alignment

3 General Alignment Scores (gap-lineal)

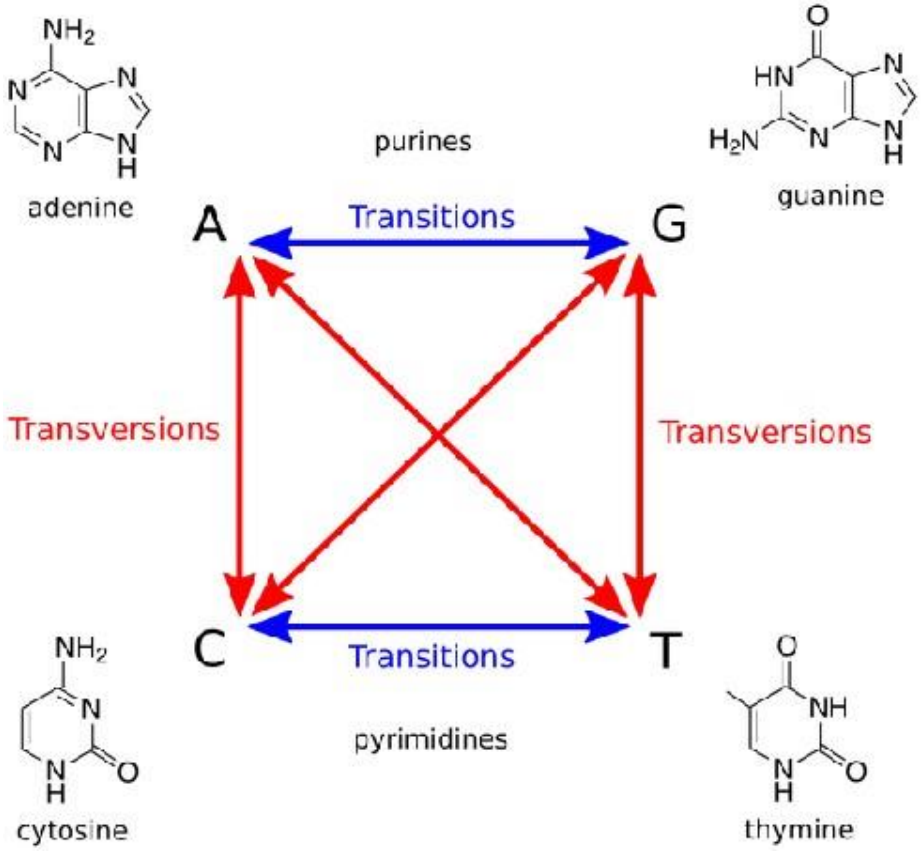


Generalizing Edit Distance

Transitions are A ↔ G and C ↔ T changes

Transversions are A — C, A — T, C — G, G — T

For random mutations, transitions should be half as frequent as transversions…



…***but*** if you compare two humans, *transition to transversion ratio* (*ti*/*tv*) is ~2.1

Generalizing Edit Distance

```
G G G T A G C G G G T T T A A C
| | | | |   | | | | | | | | | |
G G G T A A C G G G T T T A A C
```

Human substitution rate ≈ 1 in 1,000

```
G G G T A G C G G G T T T A A C
| | | | |     | | | | | | | | |
G G G T A - - G G G T T T A A C
```

Small-gap rate is ≈ 1 in 3,000

Wanted: keep basic edit distance idea and algorithm, but give different weights to different events according to likelihood

Penalty Functions and Scoring Matrices

	A	C	G	T	-
A	0	4	2	4	8
C	4	0	4	2	8
G	2	4	0	4	8
T	4	2	4	0	8
-	8	8	8	8	

2	*Transitions* (A ↔ G, C ↔ T)
4	*Transversions*
8	*Gaps*

General Alignment

Let $D[0, j] = \sum_{k=0}^{j-1} s(-, y[k])$, and let $D[i, 0] = \sum_{k=0}^{i-1} s(x[k], -)$

Otherwise, let $D[i, j] = \min \begin{cases} D[i-1, j] + s(x[i-1], -) \\ D[i, j-1] + s(-, y[j-1]) \\ D[i-1, j-1] + s(x[i-1], y[j-1]) \end{cases}$

$s(a, b)$ assigns a cost to a particular gap or substitution

$s(a, b)$:

	A	C	G	T	-
A	0	4	2	4	8
C	4	0	4	2	8
G	2	4	0	4	8
T	4	2	4	0	8
-	8	8	8	8	

2 *Transitions* (A↔G, C↔T)

4 *Transversions* (everything else)

8 *Gaps*

General Alignment (Scoring Functions)

Where do these penalty functions come from?

	A	C	G	T	-
A	0	4	2	4	8
C	4	0	4	2	8
G	2	4	0	4	8
T	4	2	4	0	8
-	8	8	8	8	

They can be based on:

- Expected frequency of the different mutational events
- How *interchangeable* are the alternatives are from a biological perspective
  - Does the substitution change the *shape* or *function* of the molecule

Prevalence of simple (linear, constant, affine) gap penalties is mostly because that's what we can do efficiently, as discussed in HW4

- One occasionally sees more general (e.g. convex) gap penalties

General Alignment (implementation)

Code

```
from numpy import zeros

def globalAlignment(x, y, s):
  """ Calculate global alignment value of sequences x and y using
  dynamic programming. Return global alignment value. """
  D = zeros((len(x)+1, len(y)+1), dtype=int)
  for j in range(1, len(y)+1):
    D[0, j] = D[0, j-1] + s('-', y[j-1])
  for i in range(1, len(x)+1):
    D[i, 0] = D[i-1, 0] + s(x[i-1], '-')
  for i in range(1, len(x)+1):
    for j in range(1, len(y)+1):
      D[i, j] = min(D[i-1, j-1] + s(x[i-1], y[j-1]), # diagonal
      D[i-1, j ] + s(x[i-1], '-'), # vertical
      D[i , j-1] + s('-', y[j-1])) # horizontal
  return D, D[len(x), len(y)]
```

Use of new penalty function

Global Alignment: Implementation

Code

```
def exampleCost(xc, yc):
  """ Cost function assigning 0 to match, 2 to transition, 4 to
    transversion, and 8 to a gap """
  if xc == yc: return 0 # match
  if xc == '-' or yc == '-': return 8 # gap
  minc, maxc = min(xc, yc), max(xc, yc)
  if minc == 'A' and maxc == 'G': return 2 # transition
  elif minc == 'C' and maxc == 'T': return 2 # transition
  return 4 # transversion
```

	A	C	G	T	-
A	0	4	2	4	8
C	4	0	4	2	8
G	2	4	0	4	8
T	4	2	4	0	8
-	8	8	8	8	

BLOSUM62

Some amino acid substitutions have a smaller impact on structure & function than others. BLOSUM62 elements are, roughly speaking, log-odds of observing these substitutions between two highly related proteins

	Ala	Arg	Asn	Asp	Cys	Gln	Glu	Gly	His	Ile	Leu	Lys	Met	Phe	Pro	Ser	Thr	Trp	Tyr	Val
Ala	4																			
Arg	-1	5																		
Asn	-2	0	6																	
Asp	-2	-2	1	6																
Cys	0	-3	-3	-3	9															
Gln	-1	1	0	0	-3	5														
Glu	-1	0	0	2	-4	2	5													
Gly	0	-2	0	-1	-3	-2	-2	6												
His	-2	0	1	-1	-3	0	0	-2	8											
Ile	-1	-3	-3	-3	-1	-3	-3	-4	-3	4										
Leu	-1	-2	-3	-4	-1	-2	-3	-4	-3	2	4									
Lys	-1	2	0	-1	-3	1	1	-2	-1	-3	-2	5								
Met	-1	-1	-2	-3	-1	0	-2	-3	-2	1	2	-1	5							
Phe	-2	-3	-3	-3	-2	-3	-3	-3	-1	0	0	-3	0	6						
Pro	-1	-2	-2	-1	-3	-1	-1	-2	-2	-3	-3	-1	-2	-4	7					
Ser	1	-1	1	0	-1	0	0	0	-1	-2	-2	0	-1	-2	-1	4				
Thr	0	-1	0	-1	-1	-1	-1	-2	-2	-1	-1	-1	-1	-2	-1	1	5			
Trp	-3	-3	-4	-4	-2	-2	-3	-2	-2	-3	-2	-3	-1	1	-4	-3	-2	11		
Tyr	-2	-2	-2	-3	-2	-1	-2	-3	2	-1	-1	-2	-1	3	-3	-2	-2	2	7	
Val	0	-3	-3	-3	-1	-2	-2	-3	-3	3	1	-2	1	-1	-2	-2	0	-3	-1	4

Amino acids

Rare; larger effect on structure/function

Common; modest effect on structure/function

negative

positive

Matrix is symmetric